Unit 1: Object-Oriented Paradigm

1.1 Features of Object-Oriented Programming (OOP)

1. **Encapsulation**:
    - **Explanation**: Encapsulation refers to the concept of bundling the data (variables) and methods (functions) that operate on the data into a single unit called a class. The idea is to hide the internal details of an object and only expose what is necessary for the outside world to interact with.
    - **Real-Life Example**: Think of a car. You interact with the car using its interface (steering wheel, pedals, etc.) without needing to understand the engine's internal mechanics. Similarly, a class hides its internal workings and only exposes the necessary methods.
2. **Abstraction**:
    - **Explanation**: Abstraction involves hiding the complexity of the system and only exposing the essential features. In OOP, this is achieved by using abstract classes or interfaces that only provide method signatures without implementations.
    - **Real-Life Example**: A TV remote is an abstraction. You don’t need to know how the remote communicates with the TV (the internal process) but only how to operate it (button press).
3. **Inheritance**:
    - **Explanation**: Inheritance allows you to define a new class based on an existing class, inheriting its properties and behaviors. This leads to code reusability and allows for creating a hierarchy of classes.
    - **Real-Life Example**: A "Dog" class could inherit from an "Animal" class, which means the dog will have all the characteristics of an animal (like eating, sleeping), but you can add dog-specific behaviors (like barking).
4. **Polymorphism**:
    - **Explanation**: Polymorphism means "many forms." It allows a single interface to be used for different data types. The two types of polymorphism are compile-time (method overloading) and runtime (method overriding).
    - **Real-Life Example**: A person can have different roles – they can be a teacher, a parent, or a student – depending on the situation. Similarly, polymorphism allows a single object to behave differently based on its context.

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIIfETBy

5. **Message Passing**:

    - **Explanation**: Objects communicate with each other using messages (method calls). This allows different objects to interact and perform tasks based on the messages they receive.
    - **Real-Life Example**: When you ask a friend to help you with something, you're sending a "message" (request), and the friend responds with an action. Similarly, in OOP, one object sends a message (method call) to another object.

1.2 Difference Between Object-Oriented and Procedure-Oriented Programming

- **Object-Oriented Programming (OOP)**:

    - OOP is based on **objects** and **classes**, where data and behavior are bundled together. It is data-centric and focuses on modeling real-world entities.
    - Example: C++, Java, Python.
    - **Advantages**: Easier to maintain and extend, better for complex systems, supports modularity, reusability, and inheritance.
- **Procedure-Oriented Programming (POP)**:

    - POP focuses on **procedures** or **functions** that operate on data. It is function-centric and treats data as separate from the functions that manipulate it.
    - Example: C, Fortran, Pascal.
    - **Advantages**: Simpler for small programs, easy to understand for procedural tasks.

Feature	OOP	POP
Concept	Focus on objects and classes	Focus on functions or procedures
Data Handling	Data is encapsulated within objects	Data is global or passed between functions
Code Reusability	High reusability via inheritance and polymorphism	Limited reusability
Modularity	Code is modular due to classes and objects	Code is modular due to functions

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy

Use Case	Complex systems, user interfaces, games	Small scripts, procedural algorithms

---

1.3 Difference Between Structure and Class

Feature	Structure	Class
Purpose	Groups data of different types together	Defines a blueprint for creating objects with data and methods
Access Specifiers	All members are public by default	Members are private by default unless specified
Inheritance	Structures do not support inheritance	Classes support inheritance (extends functionality)
Encapsulation	Does not support encapsulation	Supports encapsulation (can hide internal data)
Methods	Cannot contain functions	Can contain methods (functions)
Default Access	Public by default	Private by default

Unit 2: Introduction to C++

2.1 Data Types

- **Primitive Data Types**:
    - **int**: Integer type (whole numbers).
        - **Example**: `int age = 25;`
    - **float**: Floating-point number (decimals).
        - **Example**: `float price = 10.5;`
    - **double**: Double precision floating-point number.
        - **Example**: `double height = 6.2;`
    - **char**: Character type (single character).
        - **Example**: `char grade = 'A';`
    - **bool**: Boolean type (true or false).
        - **Example**: `bool isActive = true;`
- **Derived Data Types**:

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy

- **Array**: A collection of similar data types.
  - **Example**: `int arr[5] = {1, 2, 3, 4, 5};`
- **Pointer**: Stores memory address of a variable.
  - **Example**: `int *ptr = &age;`
- **Reference**: Refers to an existing variable.

2.2 Operators

- **Arithmetic Operators**:
  - Used for basic mathematical operations.
    - Example: `+`, `-`, `*`, `/`, `%`
- **Relational Operators**:
  - Used to compare values.
    - Example: `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical Operators**:
  - Used to combine multiple conditions.
    - Example: `&&` (AND), `||` (OR), `!` (NOT)

2.3 Conditional Statements

- **if Statement**: Used for simple conditional execution.

Example:

```
 if (age > 18) {
    cout << "Adult";
}
```

  -

- **else Statement**: Used with `if` to execute a block of code when the condition is false.

Example:

```
 if (age > 18) {
    cout << "Adult";
} else {
```

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy

```
    cout << "Not Adult";
}
```

- **switch-case Statement**: Used to select one of many code blocks to be executed.

Example:

```
 switch (day) {
    case 1:
       cout << "Monday";
       break;
    case 2:
       cout << "Tuesday";
       break;
    default:
       cout << "Invalid Day";
}
```

2.4 Loops

- **for Loop**: Used for iterating over a known number of elements.

Example:

```
 for (int i = 0; i < 5; i++) {
    cout << i << " ";
}
```

- **while Loop**: Used when the number of iterations is not known, but you need to loop until a condition is met.

Example:

```
 while (x < 10) {
    cout << x << " ";
    x++;
}
```

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy

- **do-while Loop**: Similar to the while loop, but guarantees at least one iteration.

Example:

```
do {
   cout << x << " ";
   x++;
} while (x < 10);
```

2.5 Arrays

- **Definition**: Arrays are collections of variables of the same type that are stored in contiguous memory locations.

Syntax:

```
data_type array_name[array_size];
```

Example:

```
int arr[5] = {1, 2, 3, 4, 5};
```

2.6 Input and Output Streams (cin, cout)

- **cin**: Used for taking input from the user.

Example:

```
cin >> age;
```

- **cout**: Used for displaying output to the user.

Example:

```
cout << "Hello, World!";
```

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy

-

2.7 Introduction to Namespace

- **Definition**: A namespace is a container that allows grouping of identifiers, preventing name conflicts.

Example:

```
namespace Math {
    int add(int a, int b) {
        return a + b;
    }
}
```

-

Unit 3: Classes and Objects

3.1 Specifying a Class

A class is defined using the `class` keyword, followed by the class name, and then the data members and member functions enclosed in curly braces `{}`.

Example:

```
class Car {
public:
    string model;
    void displayModel() {
        cout << "Model: " << model;
    }
};
```

-

3.2 Creating Objects

An object is an instance of a class.

Example:

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy

```
 Car car1;
car1.model = "Toyota";
car1.displayModel();
```

-

3.3 Accessing Class Members

Members of a class can be accessed using the `.` operator.

Example:

```
 car1.model = "Honda";
cout << car1.model;  // Output: Honda
```

-

3.4 Defining a Member Function Inside and Outside a Class

Inside Class:

```
 class MyClass {
public:
    void display() {
        cout << "Inside class function";
    }
};
```

-

Outside Class:

```
 class MyClass {
public:
    void display();
};

void MyClass::display() {
    cout << "Outside class function";
}
```

-

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMlIfETBy

3.5 Access Specifiers

- **public**: Members can be accessed from outside the class.
- **private**: Members cannot be accessed directly from outside the class.
- **protected**: Members can be accessed by derived classes.

Unit 4: Constructors and Destructors

4.1 Need for Constructors

- **Definition**: A constructor is a special function used to initialize an object when it is created. It is automatically invoked at the time of object creation.

4.2 Types of Constructors

1. **Default Constructor**: Takes no parameters.

Example:

```
 class Box {
public:
   Box() {
      length = 1;
      width = 1;
   }
};
```

   ○

2. **Parameterized Constructor**: Takes parameters to initialize an object with specific values.

Example:

```
 class Box {
public:
   Box(int l, int w) {
      length = l;
      width = w;
   }
};
```

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMlIfETBy

3. **Copy Constructor**: Initializes a new object as a copy of an existing object.

Example:

```
 class Box {
public:
    Box(const Box &b) {
        length = b.length;
        width = b.width;
    }
};
```

4.3 Destructors

Definition: Destructors are special functions called when an object is destroyed. They are used to release any resources allocated during the object's lifetime.

Syntax:

```
~ClassName() {
    // Cleanup code
}
```

Example:

```
 class Box {
public:
    ~Box() {
        cout << "Destructor called!";
    }
};
```

Lectures for ref:
https://www.youtube.com/watch?v=iw1Xf_33YM0&list=PLQEaRBV9gAFujcBWJhBT2XXsuMIlfETBy